

Spis treści

1. Omówienie podstaw biblioteki STL
2. Krótkie omówienie zbioru bibliotek Boost
3. Skąd pomysł na taką pracę?
4. Generalny schemat kodu
5. Przedstawienie metod, które porównujemy
6. Opis działania oraz przedstawienie wyników porównań każdej metody
7. Praktyczne zastosowania podanych metod
8. Wnioski końcowe

1. Omówienie podstaw biblioteki STL

Biblioteka Standard Template Library (STL) została stworzona w 1993 przez Alexandra Stepanova. Składa się z 4 głównych części – algorytmy, kontenery (struktury), funktory i iteratory. Została stworzona z myślą o programowaniu generycznym, w którym algorytmy operują na typach szablonowych (stąd nazwa) z pewnymi właściwościami – najczęściej typ szablonowy będący strukturą musi posiadać metody zdefiniowane przez dany iterator.

Pozwala to na dużą elastyczność, gdyż nie musimy korzystać z konkretnej struktury danych gdy korzystamy z danego algorytmu a jedynie musimy zaimplementować kilka metod w tym typie. Sprawia to, że algorytmy STL są bardzo elastyczne, a jednocześnie sprawdzanie, czy typ spełnia warunki algorytmu odbywa się w czasie kompilacji co znacząco ułatwia debugowanie.

Przykładami algorytmów z biblioteki STL są `binary_search`, `erase` czy `sort`, który będzie omówiony w tej pracy.

Kontenery to z kolei obiekty przechowujące dane podanego typu.

Użycie szablonów pozwala na przechowywanie w nich danych dowolnego typu, wliczając w to własne implementacje klas. Najpopularniejsze kontenery STL to:

`pair` (2-elementowa krotka), `vector` (dynamiczna tablica), `list` (lista dwukierunkowa), `set` (posortowany zbiór unikalnych wartości), `map` (zbiór par klucz-wartość, gdzie klucze są posortowane i unikalne). Dwie ostatnie struktury posiadają również wersje nieposortowane (`unordered_set`, `unordered_map`) które mają zamortyzowany czas stały operacji `insert/delete`.

Najczęściej używanym z nich jest `vector`, gdyż ma on czas stały dostępu do dowolnego elementu, wstawiania i usuwania elementu na końcu oraz spełnia warunki wszystkich 5 iteratorów co czyni go bardzo uniwersalnym do użycia w algorytmach biblioteki STL.

Wektor posiada swoje wady – np. nieumiejętne użycie operacji wstawiania elementu na koniec może oznaczać realokację pamięci, co wydłuża czas wstawiania n elementów do wektora z liniowego do kwadratowego. Dodatkowo przy realokacji rezerwuje on 2x więcej pamięci niż jest wymagane do przechowania elementów, co jest nieefektywne w porównaniu do zwykłej tablicy, która jest lepszym rozwiązaniem jeśli rozmiar danych jest stały.

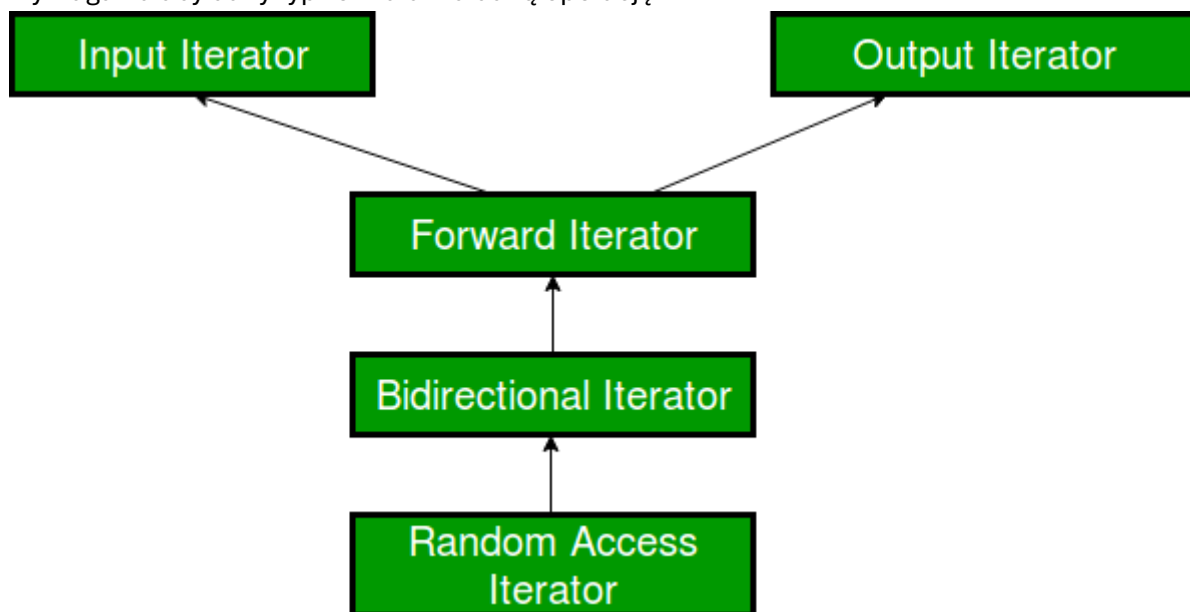
Trzecią grupą w STL są funktory – są to klasy/structy które definiują operator wywołania - `operator()(args...)`. Używane m.in. przy sortowaniu, gdzie `std::less` to funktor wywołujący porównanie $x < y$, a `std::equal_to` wywołuje porównanie $x == y$.

Funktory są przydatne gdy chcemy przekazać jakiś dodatkowy kontekst a potem na podstawie tego kontekstu wywoływać jakąś funkcję, co przedstawię jak będę omawiał `std::generate`.

Ostatnią dużą grupą STL są iteratory. Iteratory to obiekty które upraszczają interfejs, dając dostęp do kolejnych elementów według danego porządku.

Istnieje 5 typów iteratorów STL:

- a) InputIterator – iterator tylko do odczytu kolejnych wartości,
 - b) OutputIterator – iterator tylko do nadpisywania kolejnych wartości,
 - c) ForwardIterator – połączenie funkcjonalności dwóch powyższych,
 - d) BidirectionalIterator – może poruszać się zarówno od początku do końca struktury, jak i w drugą stronę, ale porusza się tylko po kolejnych wartościach, nie może „pomijać” żadnych elementów,
 - e) RandomAccessIterator – może poruszać się po dowolnych wartościach, „pomijając” elementy, co daje mu możliwości równoważne standardowemu wskaźnikowi.
- Iteratory są kluczowe przy definiowaniu algorytmów STL, gdyż pozwalają określić minimalne wymagania aby dany typ zezwalał na daną operację.



Źródło: <https://www.geeksforgeeks.org/input-iterators-in-cpp/>

Jak widać na powyższym obrazku iteratory mają pewną hierarchię.

Struktura spełniająca iterator „niżej” na obrazku może wywoływać algorytm korzystający z iteratora „wyżej” na obrazku, ale nie na odwrót.

Przykłady algorytmów, które korzystają z danego typu iteratora:

Input	find, accumulate
Output	fill, generate
Forward	replace, unique, remove_if
Bidirectional	reverse, random_shuffle
Random Access	sort, nth_element, partition

O iteratorach należy myśleć raczej jak o interfejsie niż klasie, gdyż nie definiują one same żadnej konkretnej implementacji a określają jakie warunki musi spełnić struktura aby była uznana za iterator danego typu.

2. Krótkie omówienie zbioru bibliotek Boost

Boost jest zbiorem bibliotek którego początki sięgają 1998.

Celem Boosta było ustandaryzować bibliotekę standardową oraz rozszerzyć i ulepszyć funkcjonalności biblioteki STL stąd wiele algorytmów Boosta wykorzystuje odpowiedniki z STL a z drobnymi modyfikacjami.

Boost jest zbiorem wielu różnorodnych bibliotek oferujących algorytmy w takich dziedzinach jak algebra liniowa, pseudolosowość, wielowątkowość, testowanie, operacje grafowe, operacje na plikach i wiele innych. Wiele algorytmów z Boosta nie ma odpowiednika w STL a zatem można powiedzieć, że w pewnym sensie Boost jest rozszerzeniem STL a.

3. Skąd pomysł na taką pracę?

Od kilku miesięcy jestem zatrudniony jako programista C++ i w kodzie produkcyjnym widziałem naprzemienne użycie podobnych funkcjonalności raz z STL a obok z Boosta (najczęściej dotyczyło to `shared_ptr`). Zaciekało mnie z czego to wynika i czy są jakieś istotne różnice. Zacząłem przeglądać różne metody z Boosta i szybko zauważyłem, że wiele z nich po prostu wywołuje implementacje STL a, z 1 różnicą że STL bierze strukturę jako 2 iteratory – na początek i koniec, a Boost bierze całą strukturę.

Zacząłem szukać algorytmów które mają różne implementacje i po pewnym czasie uznałem, że będzie to ciekawy i dość oryginalny temat pracy inżynierskiej, której wnioski mogłyby się komuś przydać.

4. Generalny schemat kodu

Kod jest podzielony na 3 główne grupy:

1. Algorytmy – 10 plików, w każdym są zdefiniowane 3 implementacje tego samego algorytmu – wersja STL, Boost oraz własna implementacja,
2. Struktury – własne struktury których używałem do przetestowania czasu działania algorytmów. Są to m.in. wektor liczb, losowo generowany string czy macierz,
3. Testy – 10 folderów odpowiadających każdemu algorytmowi, każdy folder ma po 2/3 pliki, które testują ten algorytm na różnych typach danych.

Algorytmy korzystają z konceptów które upewniają się, że przekazany typ danych jest poprawny dla tego algorytmu i może go przetestować.

Koncepty to dość nowa (C++ 20) technika weryfikowania, czy przekazany szablonowi typ spełnia pewne warunki. Jeśli nie to otrzymujemy błąd już na poziomie kompilacji co jest dużym ułatwieniem przy debugowaniu programu.

W większości przypadków koncept sprowadza się do wymuszenia, aby podany typ był kopiowalny, gdyż musimy przekazać te same dane wejściowe wszystkim trzem wersjom algorytmów, które je potem modyfikują w sposób nieodwracalny (np. sortowanie).

5. Przedstawienie metod, które porównujemy

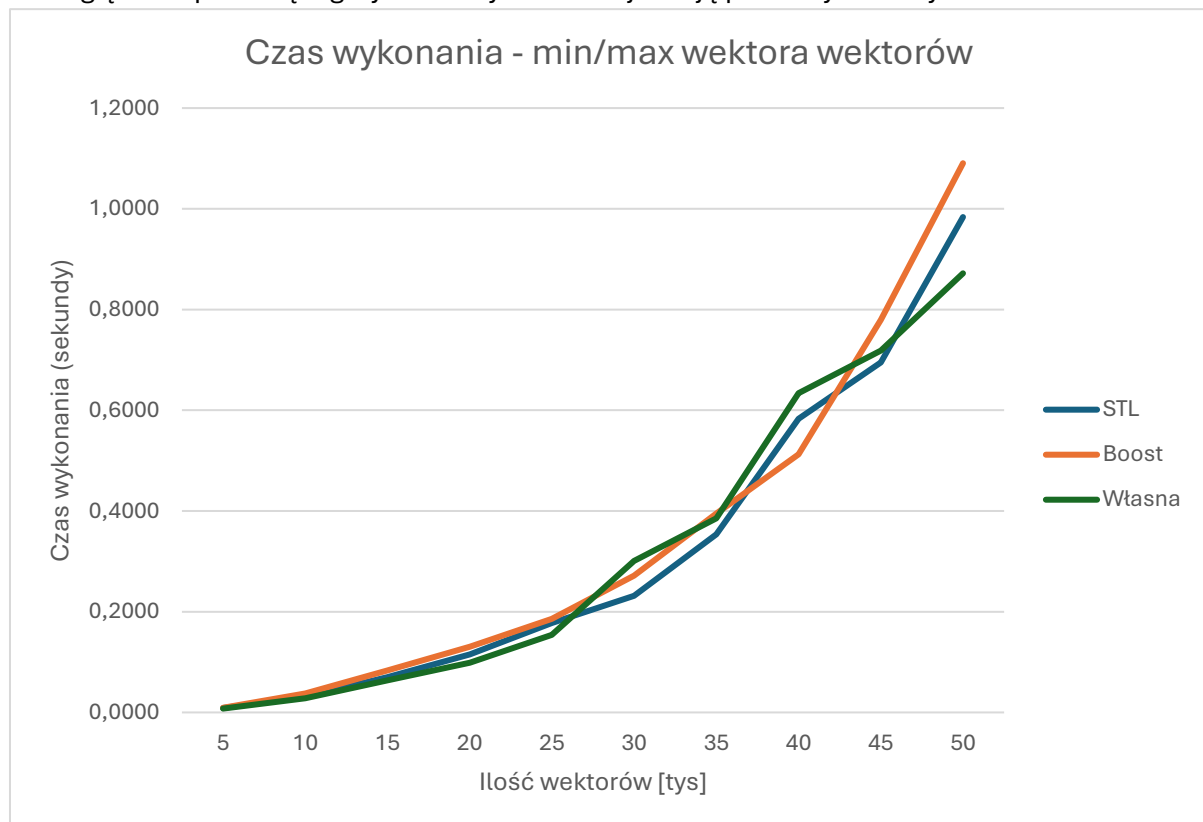
1. MinMax – znajduje 2 wartości struktury – jej minimum i maksimum.
2. Accumulate – Oblicza wynik danej operacji (suma, średnia, minimum, maksimum) zaaplikowanej do wszystkich elementów struktury.
3. Merge – scala 2 posortowane struktury w 1 posortowaną strukturę.
4. Sort – sortowanie niestabilne elementów.
5. Transform – przekształca strukturę wejściową w strukturę innego typu.
6. Unique – filtruje strukturę wejściową tak, że na wyjściu dla każdej pary sąsiednich elementów x, y zachodziło $x \neq y$
7. N-th element - przesuwa n-ty (najmniejszy) element na n-tą pozycję. Wszystkie element mniejsze od niego będą miały mniejszy indeks, a większe będą miały większy indeks. Nie sortuje ani nie zachowuje kolejności elementów wewnątrz tych 2 podgrup.
8. Regex – zwraca wektor wszystkich dopasowań do wzorca.
9. Remove Erase if – usuwa wszystkie elementy, które spełniają dany predykat.
10. Generate – uzupełnia strukturę kolejnymi wywołaniami operatora().

6. Opis działania oraz przedstawienie wyników porównań każdej metody

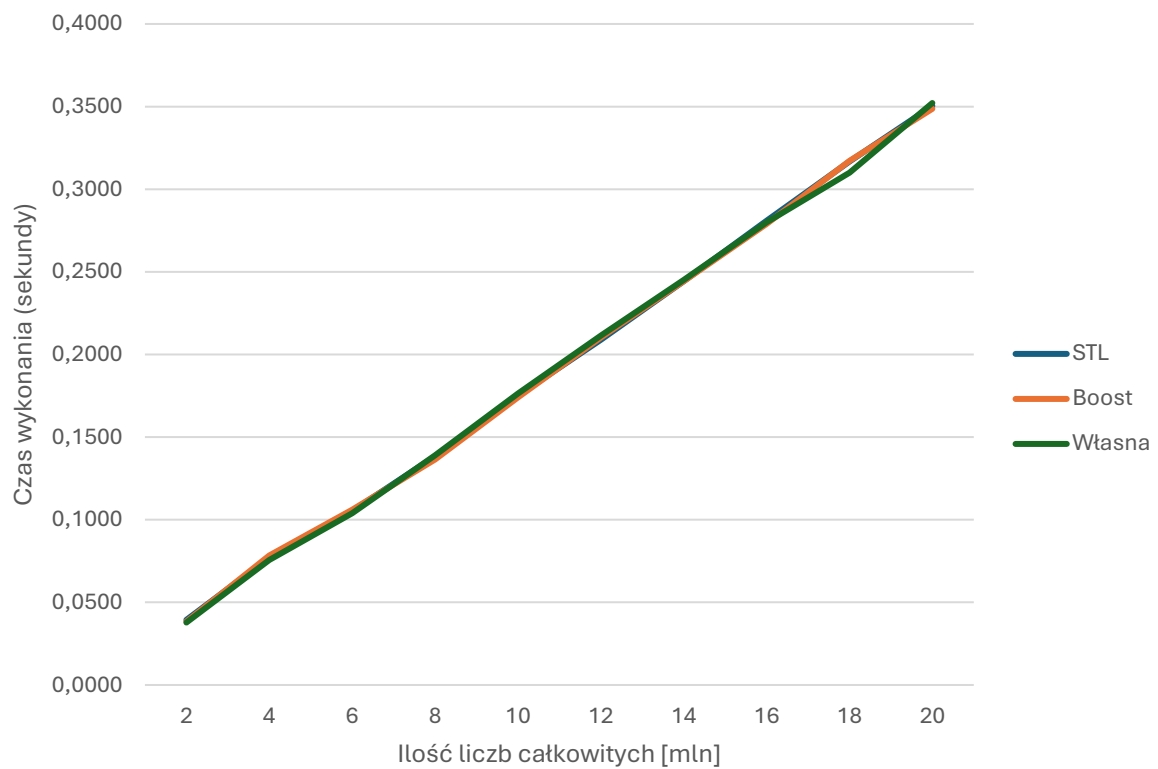
1. MinMax – znajduje 2 wartości struktury – jej minimum i maksimum.

Wszystkie wersje algorytmu działają podobnie - iterują po wszystkich elementach struktury, porównując każdy z nich z dotychczasowym minimum i maksimum.

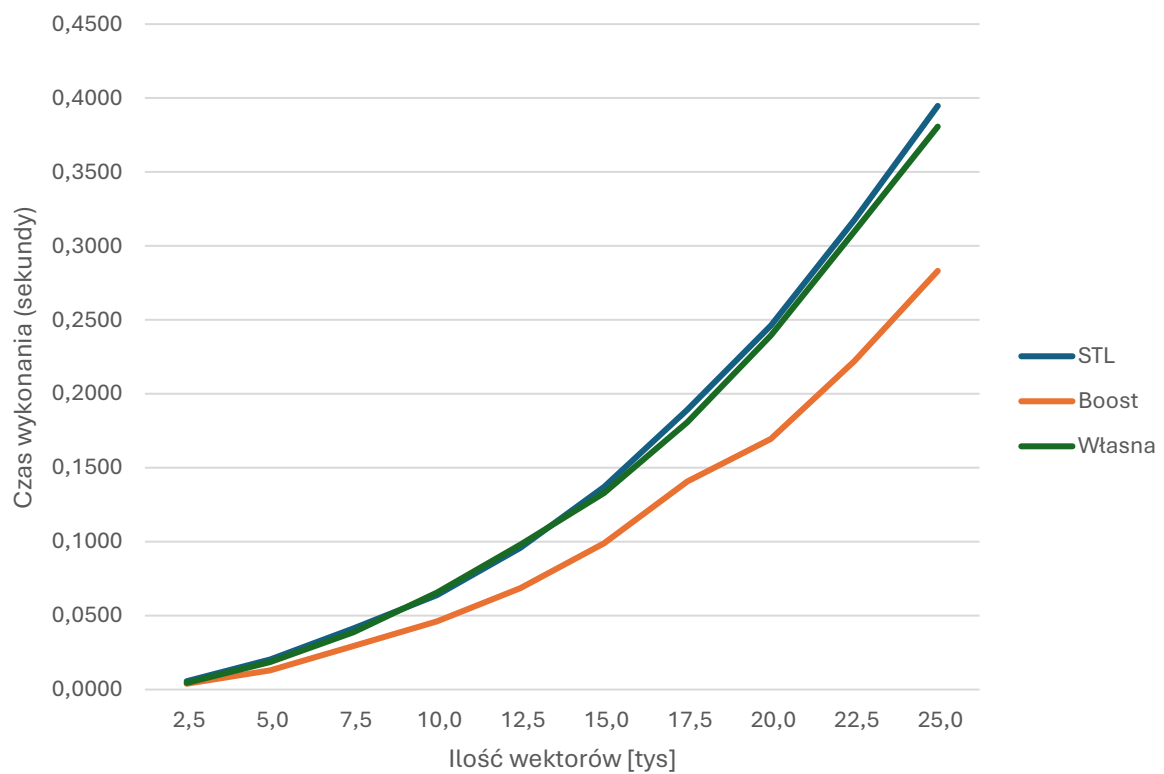
Ze względu na prostotę algorytmu wszystkie wersje mają podobny czas wykonania testów.



Czas wykonania - min/max liczb całkowitych na zbiorze



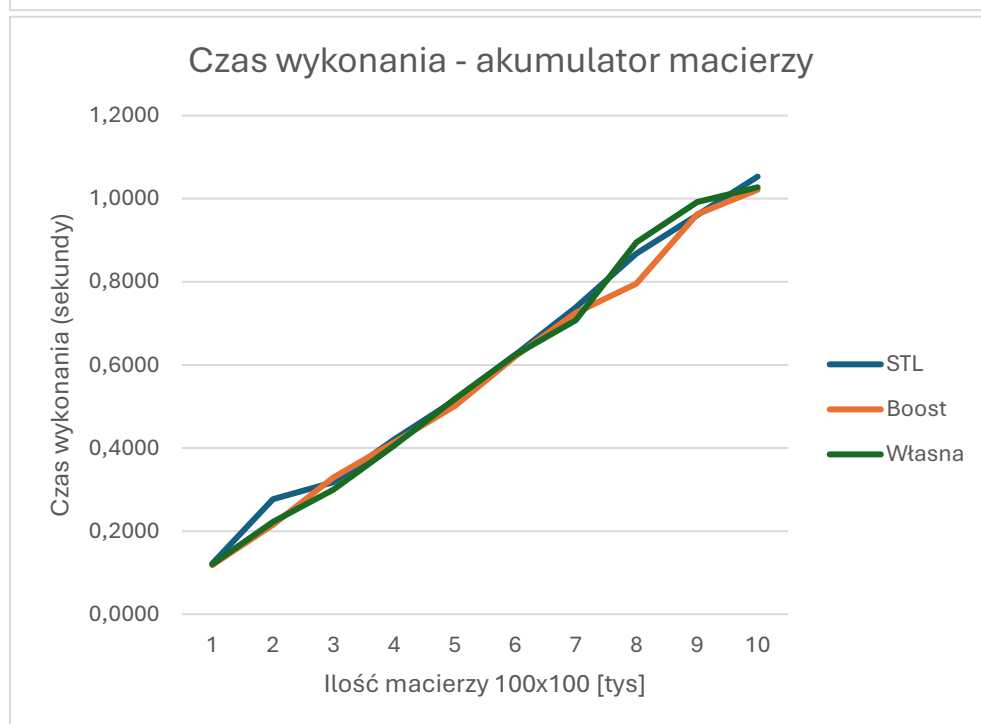
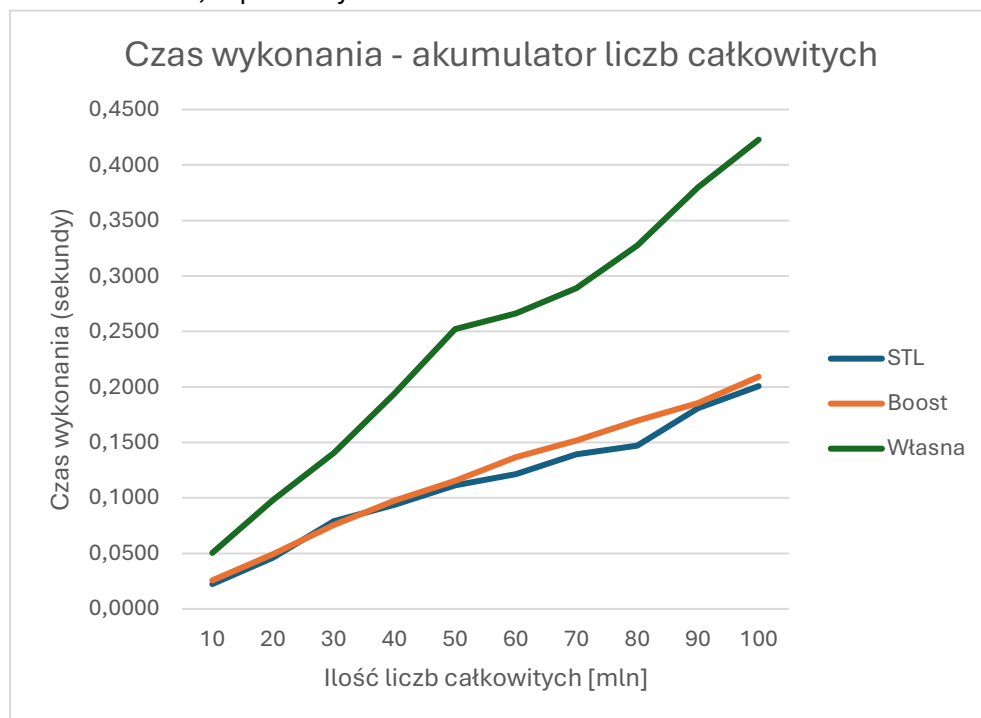
Czas wykonania - min/max zbioru wektorów



2. Accumulate – Oblicza wynik danej operacji (suma, średnia, minimum, maksimum) zaaplikowanej do wszystkich elementów struktury.

Typ elementu oprócz kopiowalności musi też być możliwy do stworzenia konstruktorem domyślnym (który oznacza „zero” dla danego typu) co jest weryfikowane za pomocą konceptu. Wszystkie wersje wyliczają na początek sumę, później minimum i maksimum, a na koniec na podstawie sumy wyliczają średnią.

Czasy wywołania różnią się w zależności od dobranych danych – dla liczb całkowitych wynik własnej implementacji odbiegał od reszty podczas gdy dla par liczb odbiegał on dla wersji STL. Dla testów macierzy (wektora wektorów) wszystkie 3 metody wykonywały się, zgodnie z oczekiwaniami, w podobnym czasie.

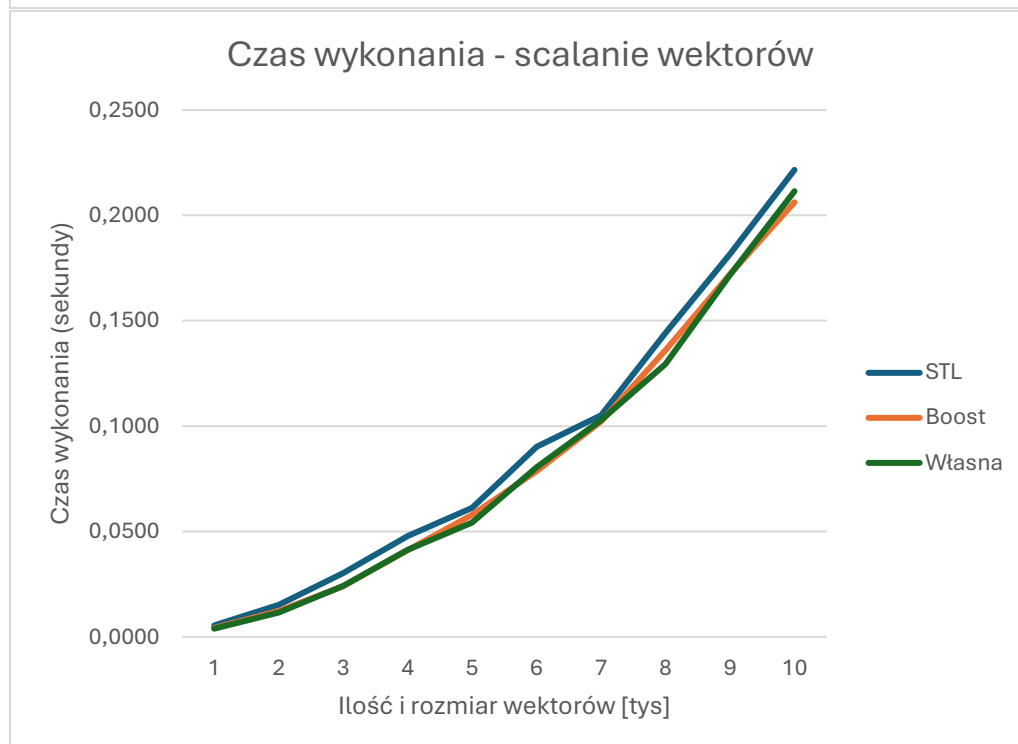
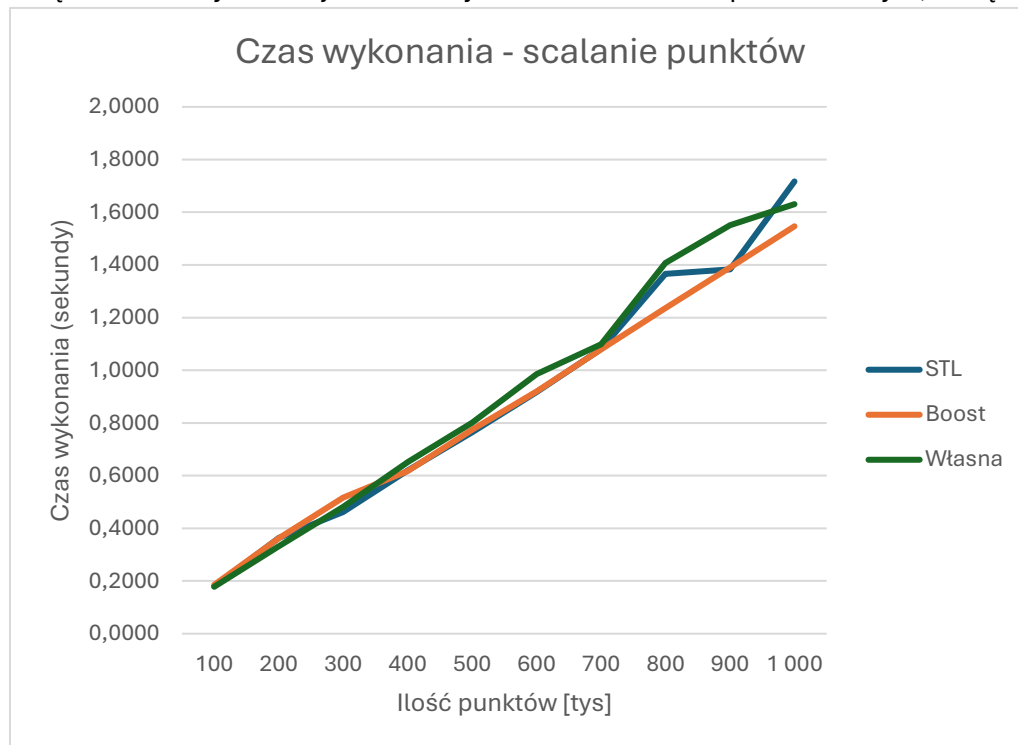


3. Merge – scala 2 posortowane struktury w 1 posortowaną strukturę.

STL i Boost korzystają z tego samego algorytmu i różnią się ilością argumentów – Boost oczekuje na wejściu 3 argumentów – 2 struktury z dostępem losowym oraz iterator na początek struktury wynikowej, a potem przekazuje wersji STL te argumenty rozbijając struktury na 2 iteratory – na początek i koniec.

Oba algorytmy korzystają ze zoptymalizowanej wersji scalania poprzez porównywanie kolejnych elementów obu struktur.

Moja implementacja jest identyczna, stąd bardzo podobne czasy wywołania testów. Jednakże zwężłość i elastyczność jest na korzyść STL/Boost zatem powiedziałbym, że są one lepsze.

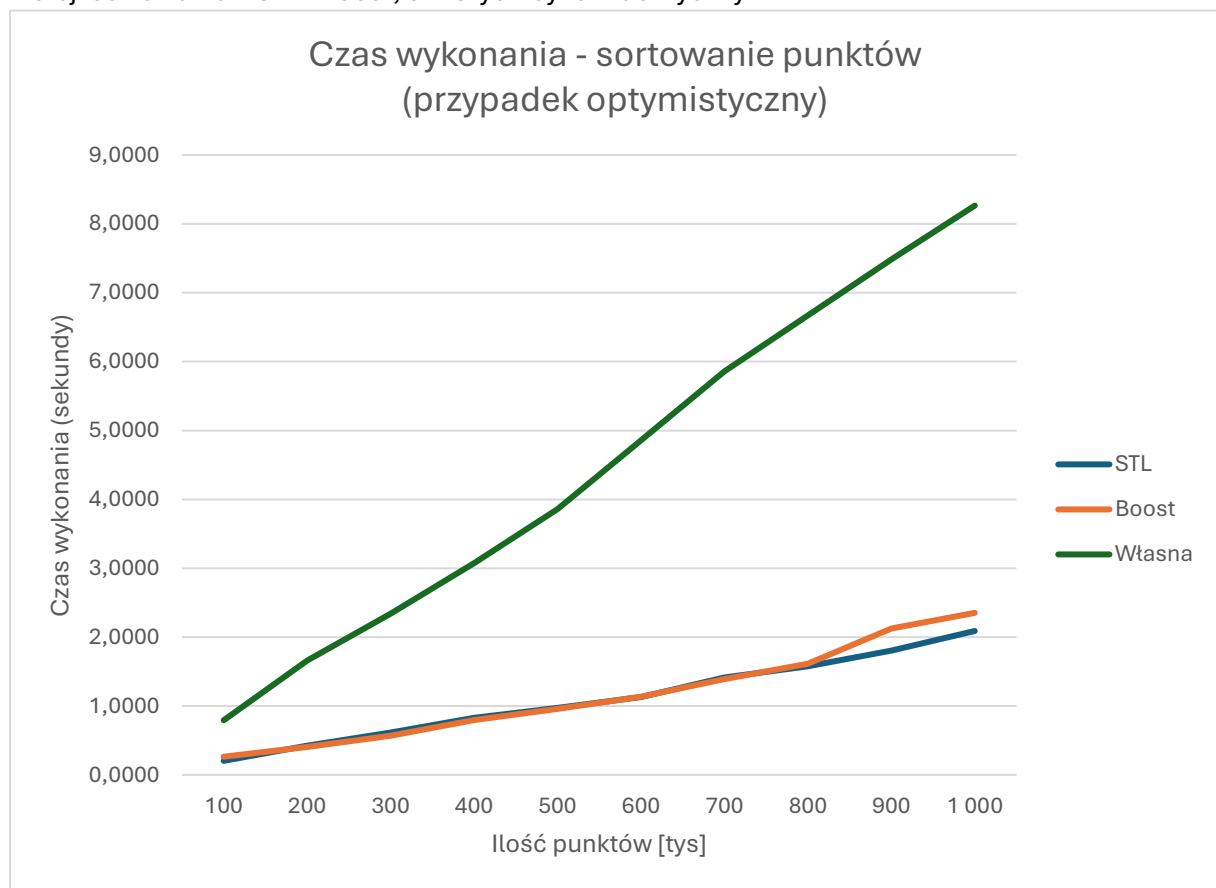


4. Sort – sortowanie niestabilne elementów.

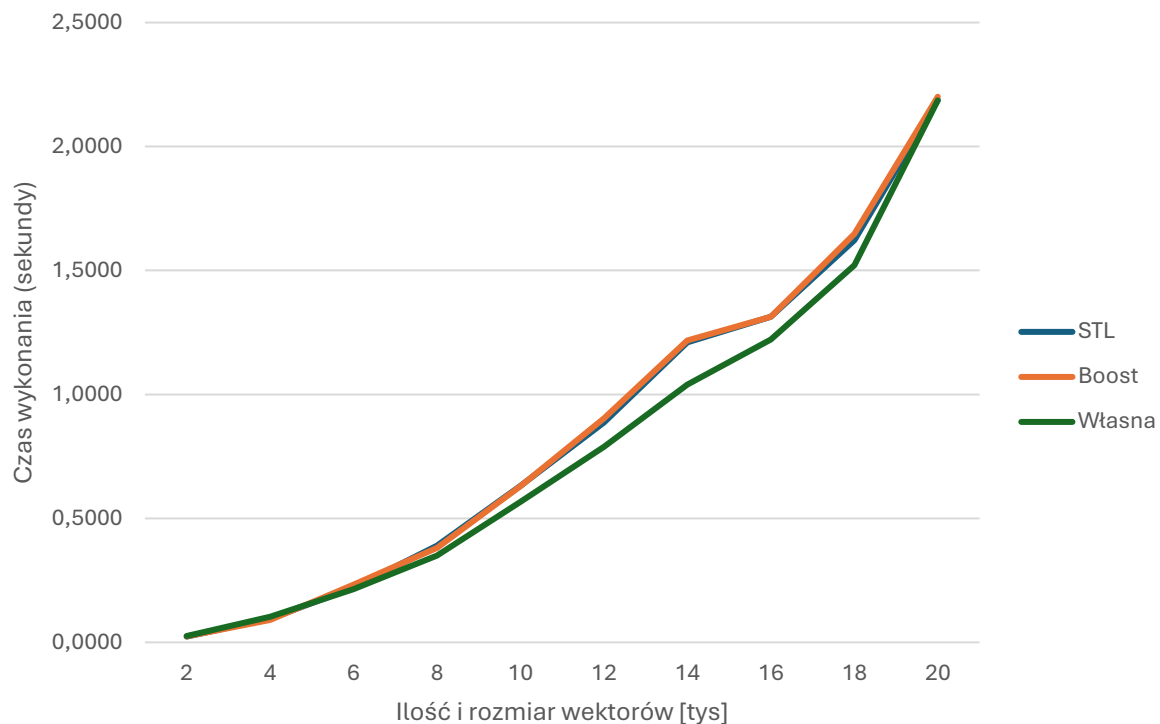
STL i Boost korzystają z tego samego algorytmu i różnią się jedynie detalem – Boost oczekuje na wejściu 1 argumentu, struktury z dostępem losowym (dostępem do każdego elementu w dowolnym momencie) a potem przekazuje wersji STL 2 argumenty – iterator na początek tej struktury i drugi na jej koniec.

Oba algorytmy korzystają z introsorta, który jest połączeniem quicksorta, heapsorta i insert-sorta: jeśli jest więcej niż 16 elementów to wykonuje $2 \cdot \log(n)$ rekurencyjnych wywołań quicksorta, jeśli przy ostatnim z nich nadal jest więcej niż 16 elementów to wywołuje na nich heapsort. Po wykonaniu quicksorta na tych ponad 16-elementowych podgrupach dokonuje się insertion sort na pozostałych, nieposortowanych grupach o liczebności do 16 elementów. Quicksort do znalezienia pivotu wykorzystuje medianę 3 elementów – pierwszego, środkowego i ostatniego.

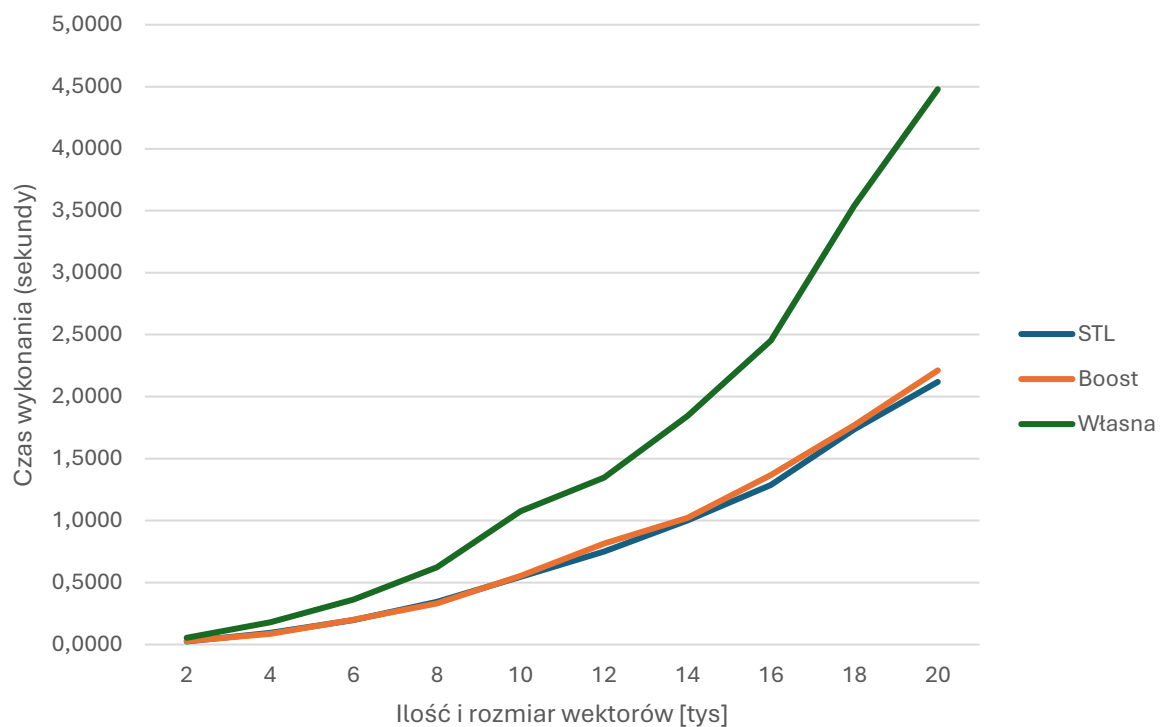
Moja implementacja była podobna, również korzystała z quicksorta z medianą trzech i insert-sorta dla małych danych, ale nie korzystała z heapsorta w przypadku wielu wywołań quicksorta, co było widoczne w czasie działania tej funkcji – rósł on szybciej niż w przypadku 2 pozostałych wersji sortowania z STL i Boost, u których był on identyczny.



Czas wykonania - sortowanie wektorów
(przypadek optymistyczny)



Czas wykonania - sortowanie wektorów
(przypadek pesymistyczny)

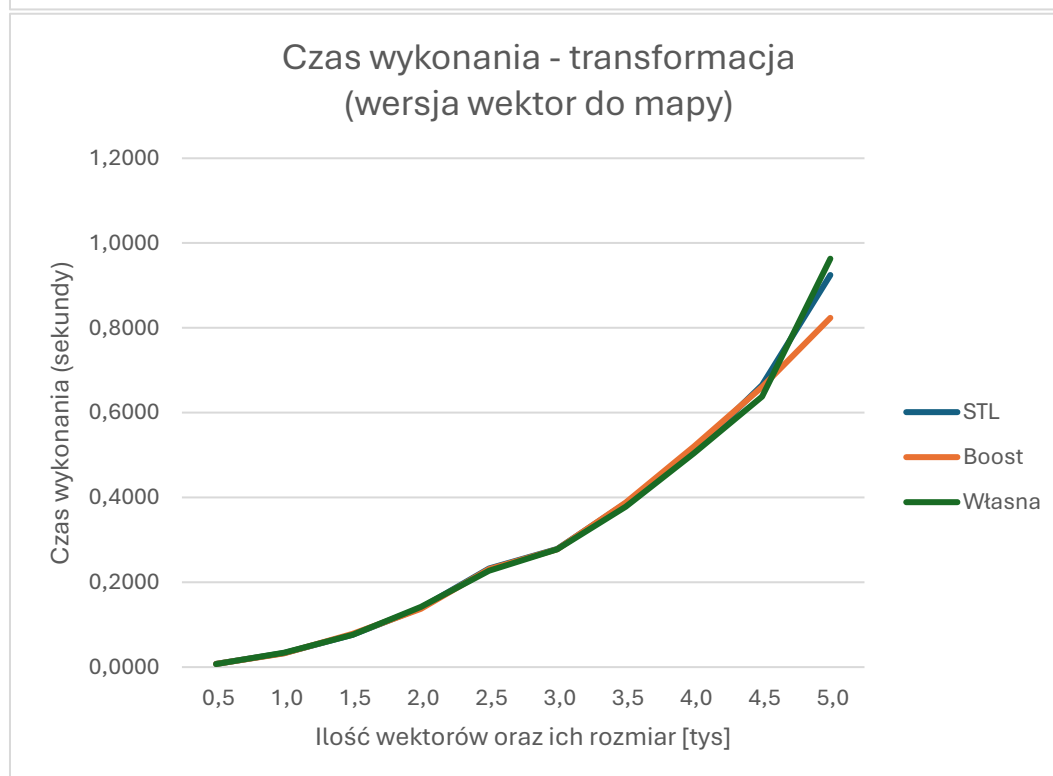
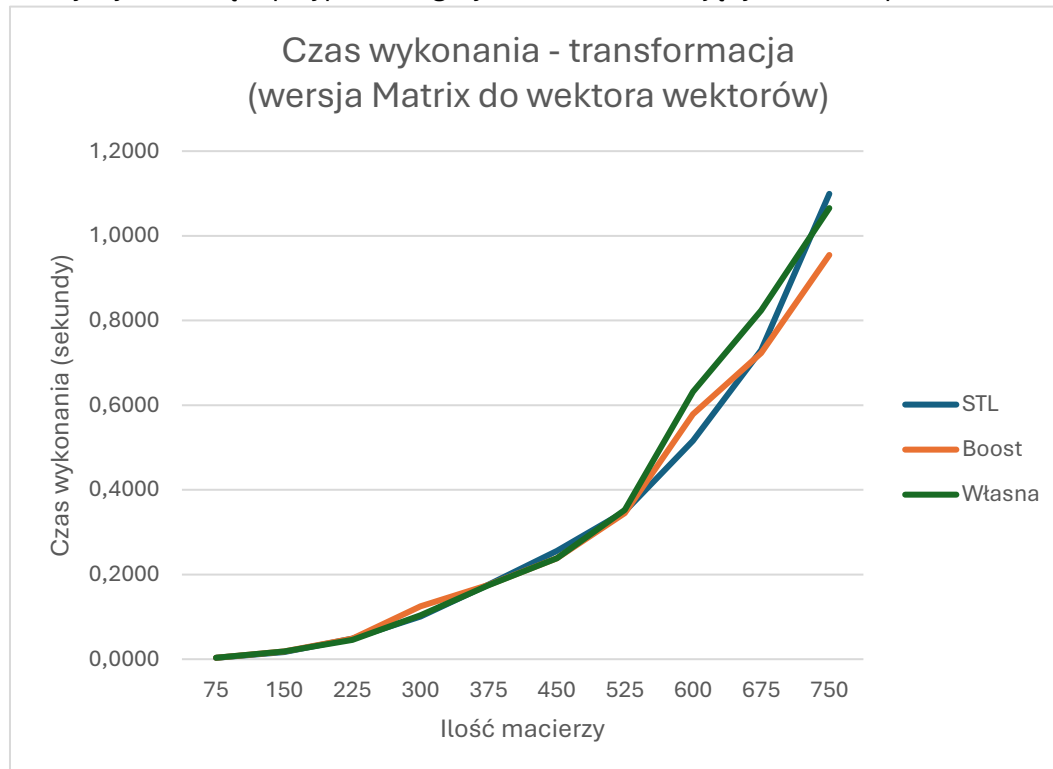


5. Transform – przekształca strukturę wejściową w strukturę innego typu.

Zarówno w wersji STL jak i Boost korzystamy z `std::views::transform`, który zwraca zakres wynikowy. Zakres ten przenosimy do wektora żeby otrzymać stabilną strukturę z łatwym dostępem.

W mojej własnej implementacji skorzystałem z pętli `for-each`, w której do wektora wynikowego dodaje wywołania transformacji kolejnych elementów.

Czasy wywołań są w przypadku algorytmów transformujących bardzo podobne.



6. Unique – filtruje strukturę wejściową tak, że na wyjściu dla każdej pary sąsiednich elementów x, y zachodziło $x \neq y$.

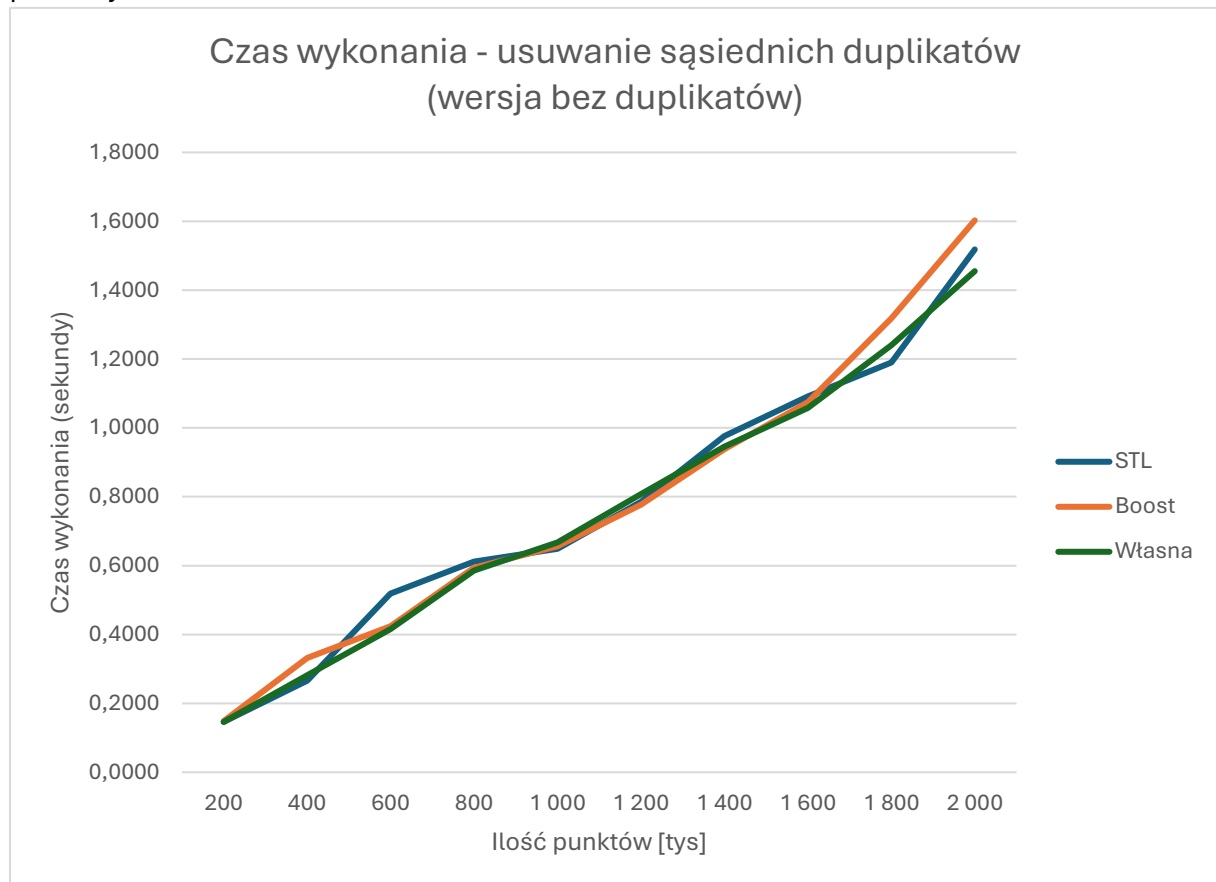
W wersji STL korzystam z `std::ranges::unique`, która zwraca zakres do usunięcia, który następnie jest używany w metodzie `erase(beginIter, endIter)`.

W wersji Boost używam `boost::adaptors::uniqued`, który zwraca zakres wynikowy (czyli odwrotnie niż w przypadku STL). Zakres ten należy skopiować do wektora żeby otrzymać stabilną strukturę z łatwym dostępem.

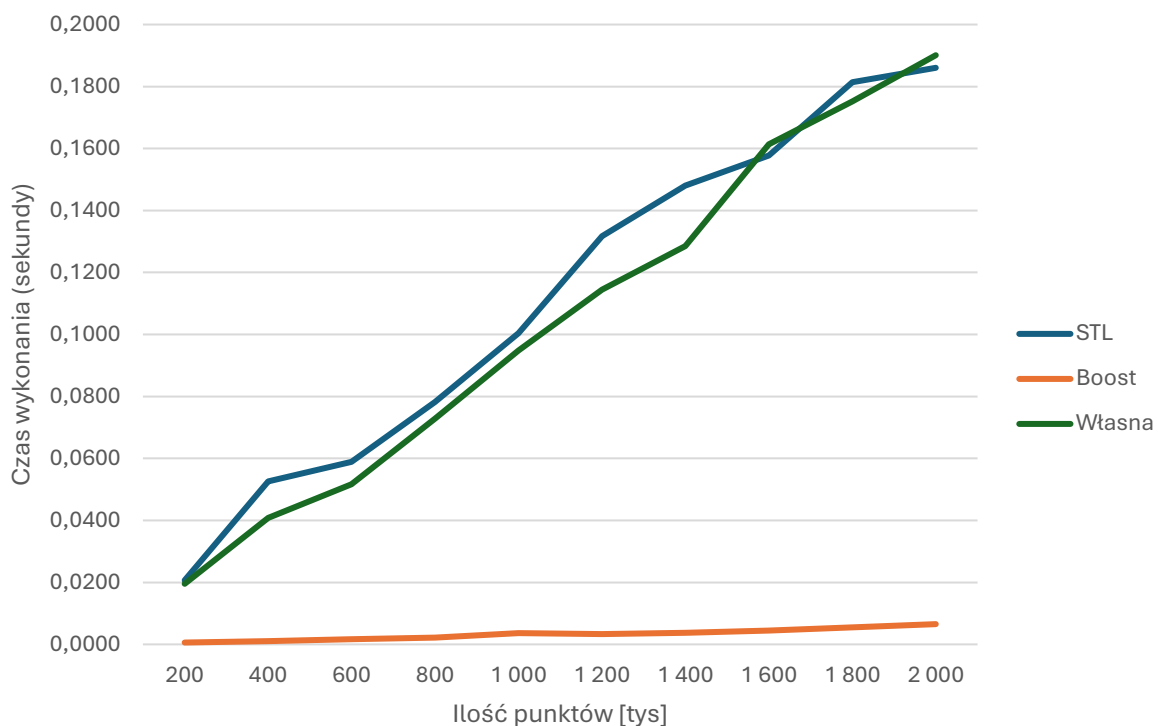
Z kolei mój algorytm wyszukuje pierwszy indeks i , dla którego $x[i-1] == x[i]$ i przesuwa wszystkie następujące elementy o $k-1$ miejsc w lewo, gdzie k to ilość kolejnych wystąpień duplikatu.

Testy były podzielone na 2 główne grupy – brak duplikatów oraz same duplikaty.

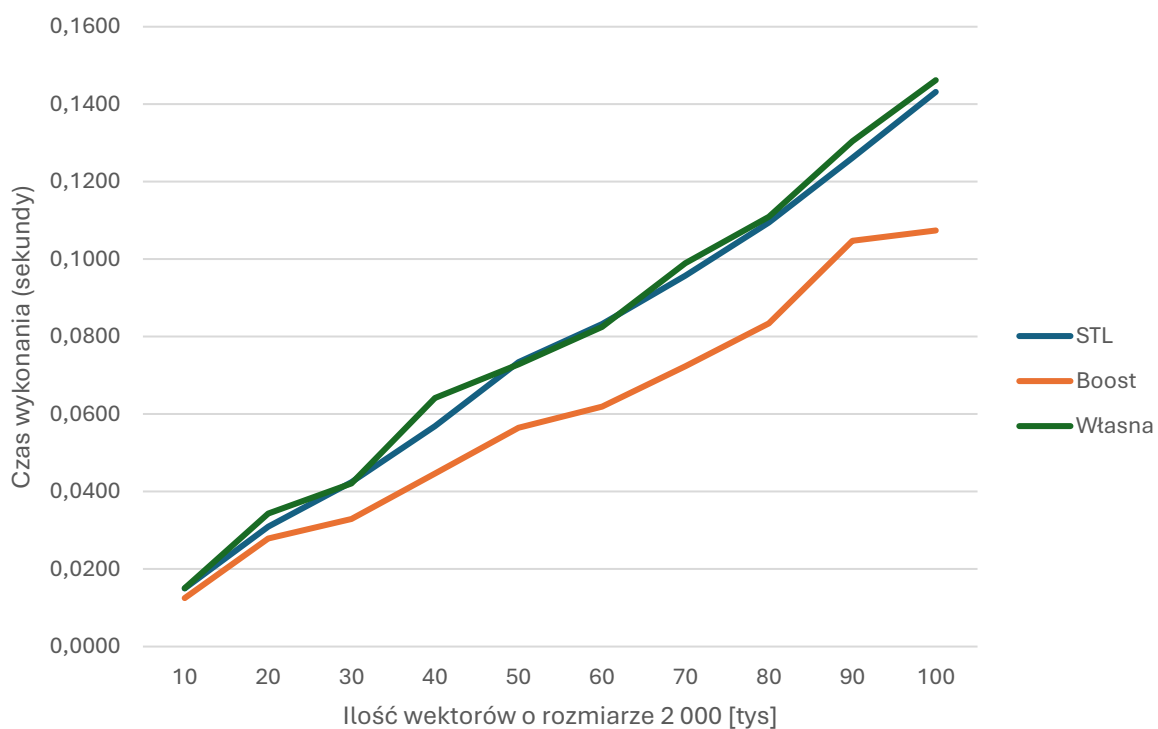
W pierwszym przypadku czasy wszystkich 3 wersji były podobne, w drugim przypadku Boost miał wyraźnie lepszy czas (zwłaszcza w przypadku par liczb) od STL'a i mojej wersji, które osiągnęły podobny czas.



Czas wykonania - usuwanie sąsiednich duplikatów
(wersja z samymi duplikatami)



Czas wykonania - usuwanie sąsiednich duplikatów
(wersja z samymi duplikatami)

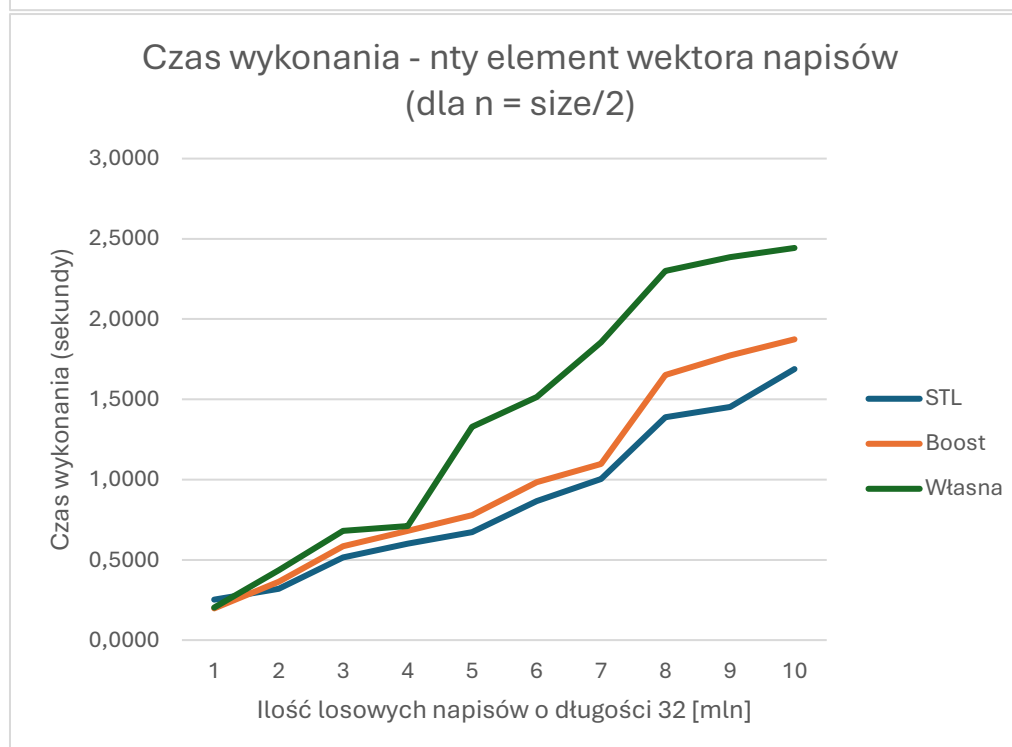
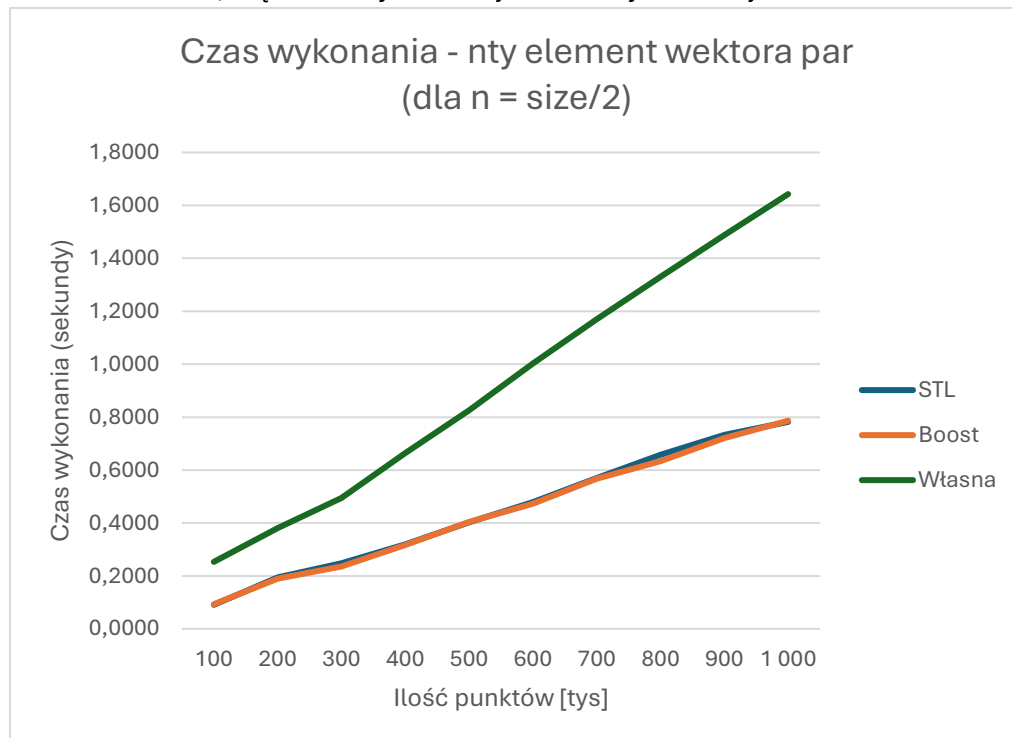


7. N-th element - przesuwa n-ty (najmniejszy) element na n-tą pozycję.

STL i Boost korzystają z tego samego algorytmu i różnią się ilością argumentów – Boost oczekuje na wejściu 2 argumentów – struktury z dostępem losowym oraz indeksu który ma być wyszukany, a potem przekazuje wersji STL te argumenty rozbijając strukturę na 2 iteratory - początek i koniec.

Algorytm korzysta z introselecta, który działa bardzo podobnie co introsort z pkt 4 z tą różnicą, że tutaj granicą jest 3 zamiast 16.

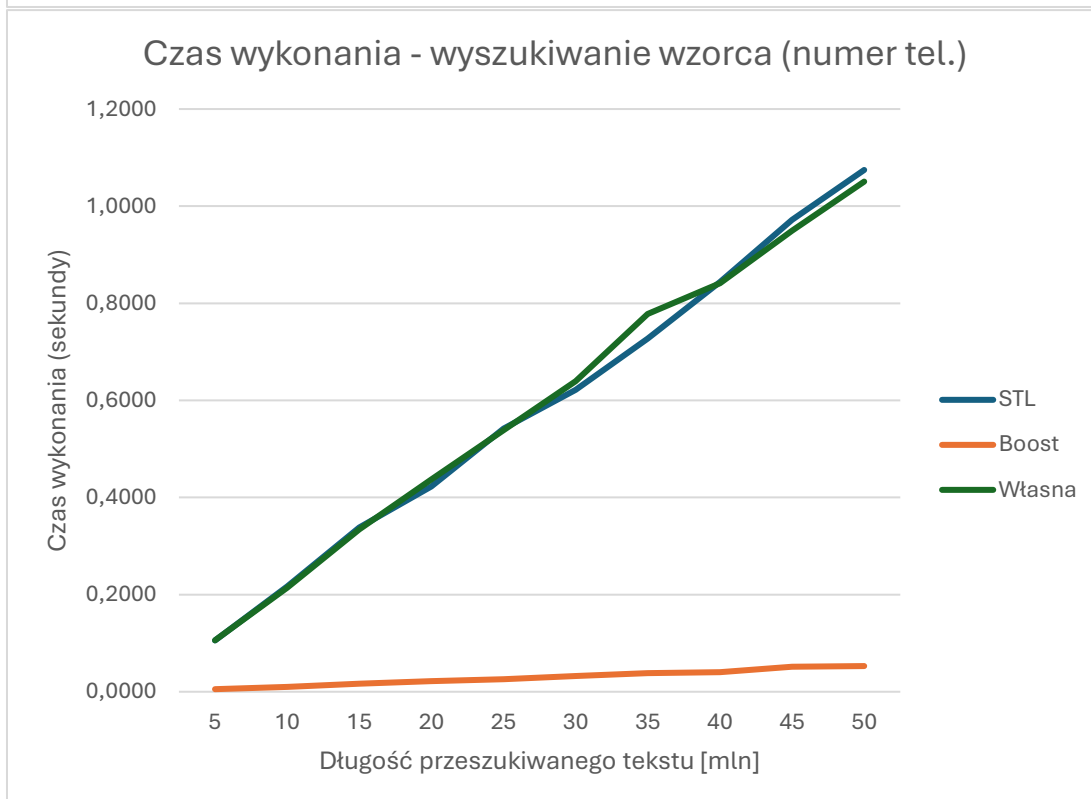
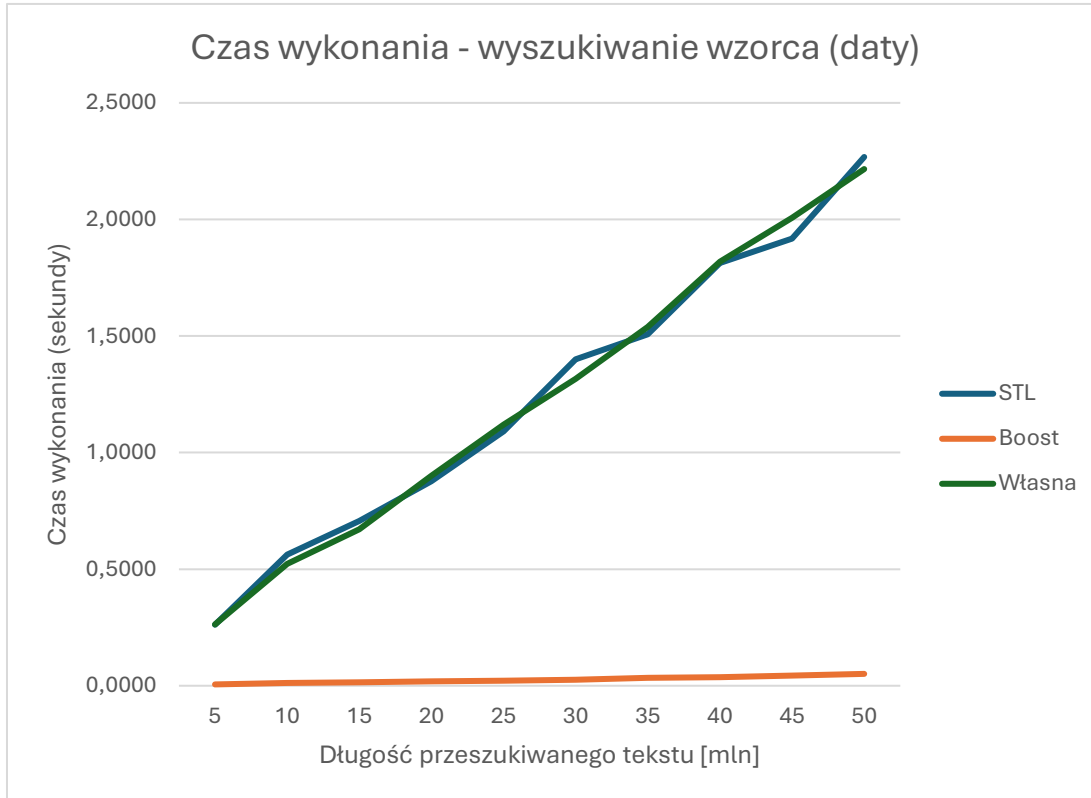
Moja implementacja również jest podobna do tej z pkt 4 zatem posiada podobne niedoskonałości, stąd czas wywołania jest dłuższy niż wersji STL/Boost.



8. Regex – zwraca wektor wszystkich dopasowań do wzorca.

Boost korzysta ze zoptymalizowanego względem STL algorytmu opartego na maszynie stanów, co sprawia, że jego czas wywołania rośnie wolniej i wykonuje mniej operacji backtrackingu.

Moja implementacja korzysta z wielokrotnego wyszukiwania kolejnego wzorca na pozostałym tekście, co sprawia, że ma czas wywołania podobny do wersji STL i dużo gorszy niż wersja Boost.



9. Remove Erase if – usuwa wszystkie elementy, które spełniają dany predykat.

STL wykonuje iterację po kolejnych elementach usuwając te, które spełniają predykat.

Na koniec zwraca liczbę usuniętych elementów.

Boost działa podobnie, tyle że najpierw wywołuje `std::remove_if` która przesuwa wszystkie spełniające predykat elementy na miejsce wcześniejszych elementów które go nie spełniały.

To sprawia, że po wykonaniu `std::remove_if` mamy k ostatnich elementów w stanie nieokreślonym, gdzie k to ilość elementów które spełniało predykat (zostały usunięte).

nr operacji	x1	x2	x3	x4	x5	x6	x7
1	1	2	3	5	6	7	9
2	1	2	3	5	6	7	9
3	1	3	?	5	6	7	9
4	1	3	5	?	6	7	9
5	1	3	5	7	6	?	9
6	1	3	5	7	9	?	?

Niech będzie wektor { 1, 2, 3, 5, 6, 7, 9 } oraz predykat $x \% 2 == 0$.

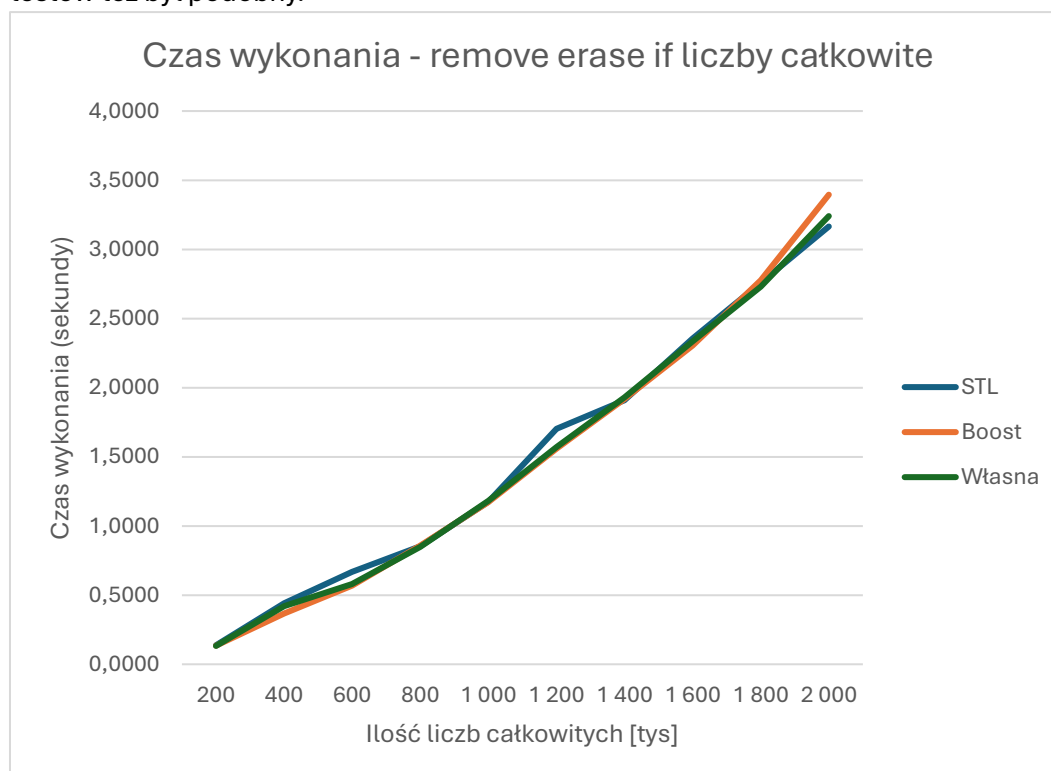
`std::remove_if` usunie 2 elementy (2, 6) poprzez przesunięcie 7 i 9 „na przód” wektora.

Operacja ta zachowa kolejność elementów dla których predykat nie jest spełniony oraz (co ważne, a niektórzy zapominają!) zachowa początkowy rozmiar wektora z ostatnimi elementami w stanie nieokreślonym. Operacja zwróci iterator na pierwszy element spełniający predykat (tutaj pierwsza wartość nieokreślona ?) lub na iterator `end()` jeśli taki nie istnieje.

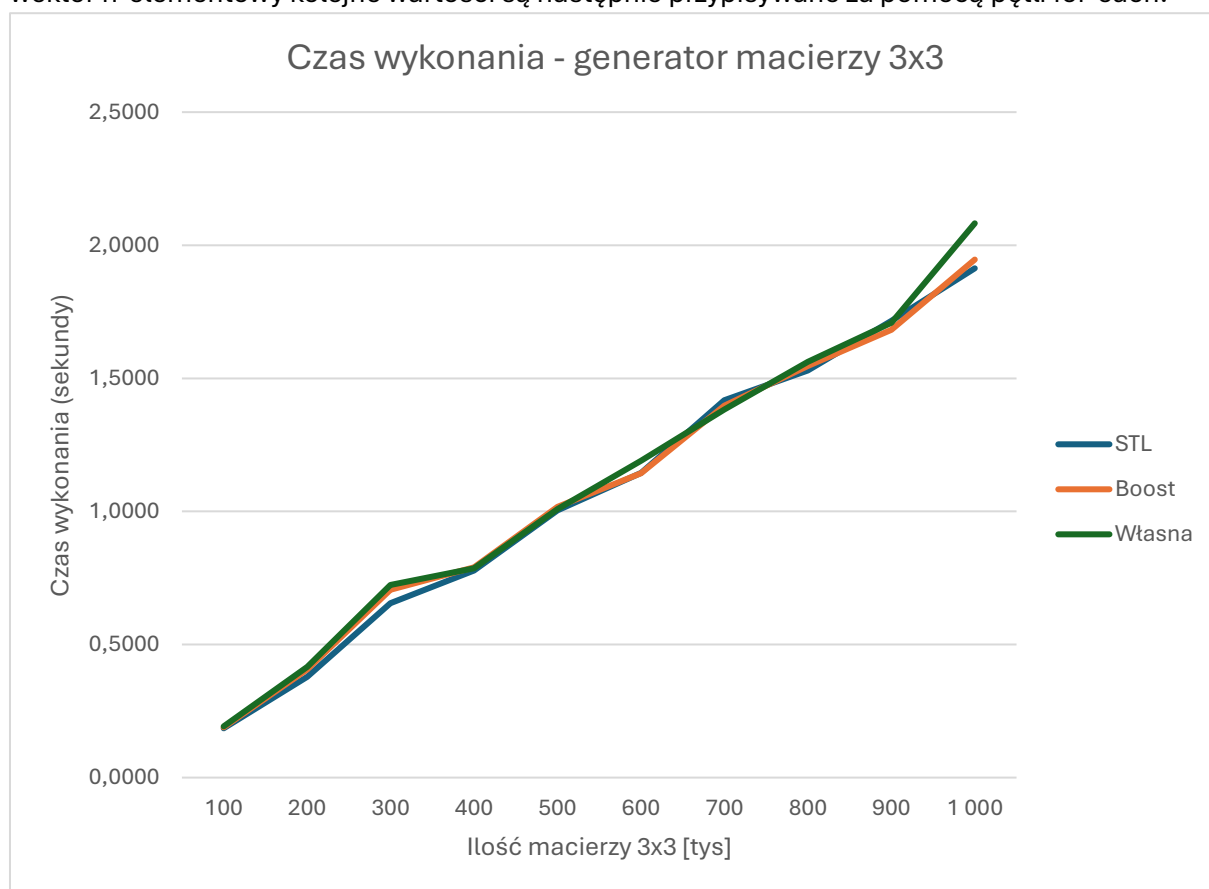
Po zakończeniu operacji `std::remove_if` musimy usunąć ostatnich k elementów które albo spełniają predykat albo są w stanie nieokreślonym. Robimy to za pomocą operacji `erase(startIter, endIter)` która usuwa wszystkie elementy w zakresie `[startIter, endIter)`.

Moja implementacja jest bardzo podobna do tej Boosta, również polega na przesuwaniu elementów spełniających predykat na przód oraz usunięciu ostatnich elementów.

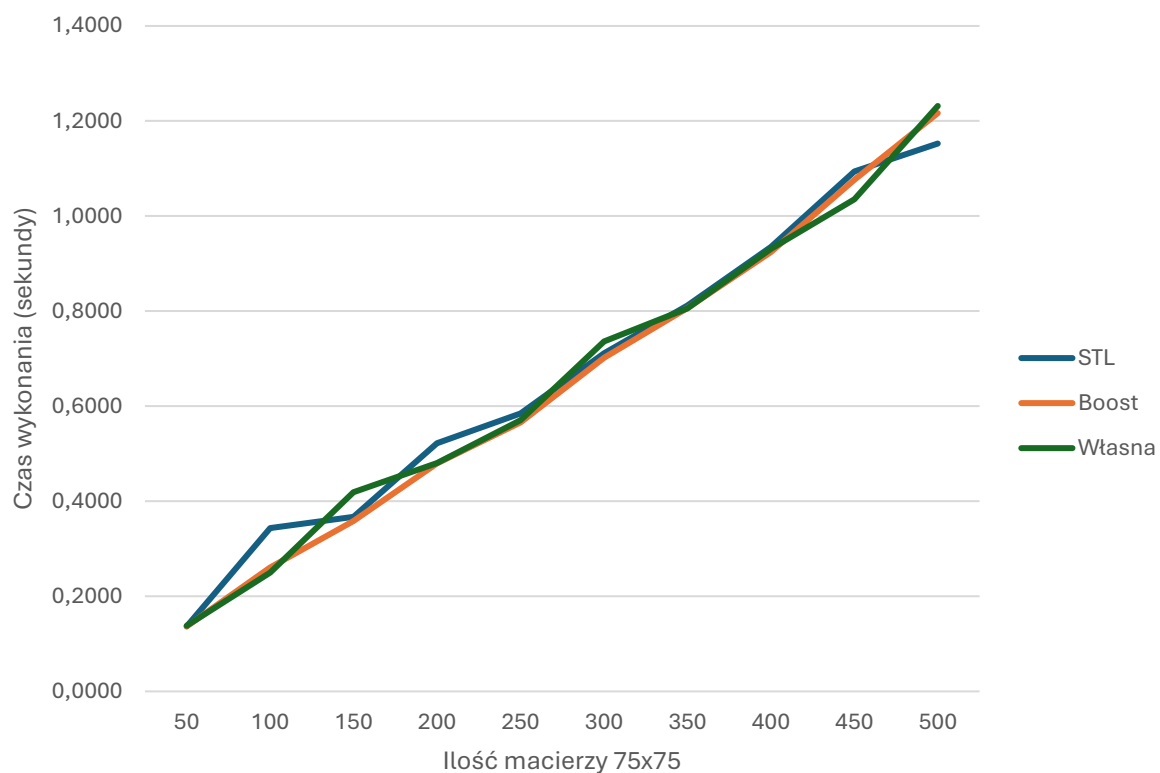
Jako że wszystkie 3 wersje algorytmu bazowały na dość podobnym pomysłe to czas wykonania testów też był podobny.



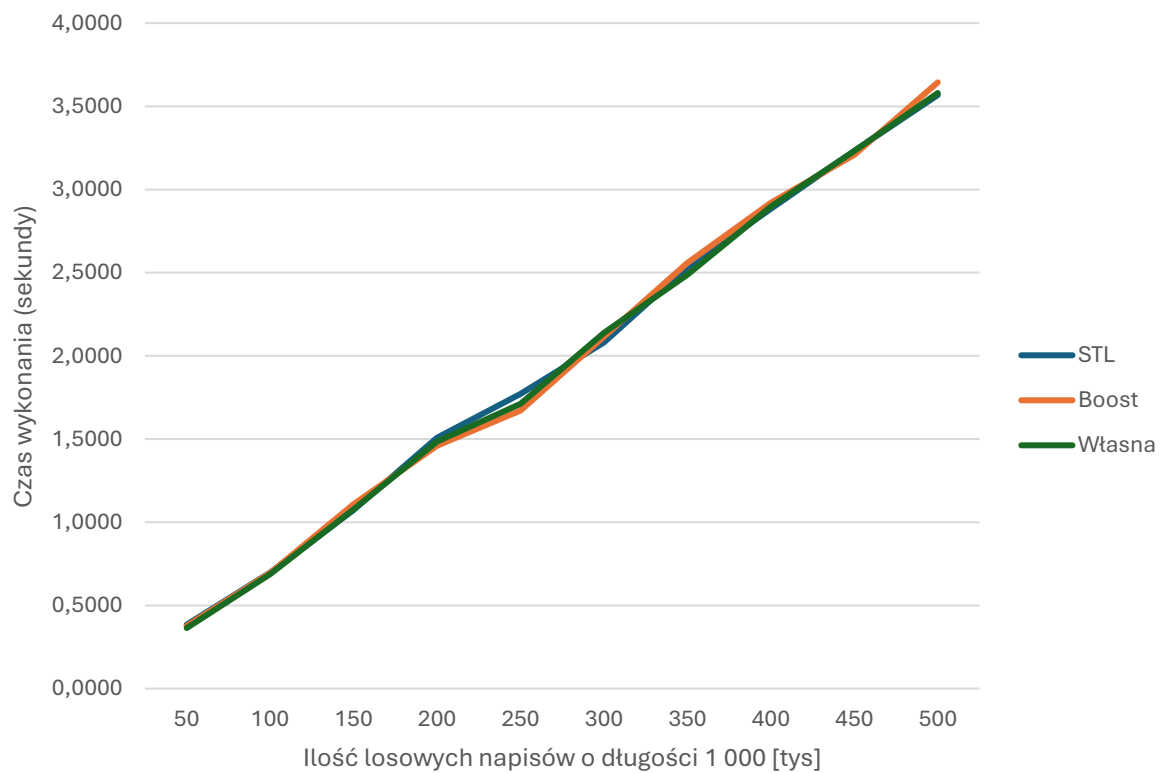
10. Generate – uzupełnia strukturę kolejnymi wywołaniami bezparametrowego generatora. Wszystkie implementacje przekazywały jako generator funkcję create, która była bezparametrowym dekoratorem pozwalającym na wywołania właściwego generatora, który przyporządkowywał wartość $i+1$ szemu elementowi na podstawie wartości poprzednika. Zatem w testach należało przekazać nie tylko funkcję generującą elementy na podstawie wartości poprzednich, ale też wartość początkową oraz ilość elementów do wygenerowania. Boost oczekuje struktury wejściowej z elementami typu T oraz generatora, czyli bezparametrowej funkcji zwracającej element typu T. Następnie wywołuje implementację STL z iteratorami na początek i koniec i dokonuje iteracji po wszystkich elementach [startIter, endIter), przypisując każdej wartości w zakresie wynik wywołania generatora. Jako że jest to bardzo intuicyjny sposób działania to moja implementacja, która była ograniczona do wektorów, zachowywała się bardzo podobnie. Po zarezerwowaniu miejsca w pamięci na wektor n-elementowy kolejne wartości są następnie przypisywane za pomocą pętli for-each.



Czas wykonania - generator macierzy 75x75



Czas wykonania - generator losowych napisów



7. Praktyczne zastosowania podanych metod

Cała ta praca miała na celu przedstawić przydatne w życiu codziennym programisty C++ algorytmy aby rozumiał, jak one działają i pokazać, że z reguły warto korzystać z gotowych implementacji STL/Boosta. Poniżej przedstawię po 3 konkretne przykłady kiedy dany algorytm może okazać się użyteczny.

1. MinMax

Jednoczesne szukanie wartości minimalnej i maksymalnej przydaje się w wielu sytuacjach, zwłaszcza w dziedzinie analizy danych, m.in. do określenia zakresu wartości akcji danej firmy na przestrzeni ostatnich tygodni/miesięcy. Wiedza o ekstremach nie jest wystarczająca do podjęcia decyzji kiedy kupić a kiedy sprzedać akcje, ale na pewno ułatwia podjąć taką decyzję.

Inną sytuacją jest ocena wyników osiąganych przez sportowców, zwłaszcza w sportach zespołowych. Znając ekstrema danych parametrów z obecnego i poprzedniego sezonu jesteśmy w stanie oszacować w jakiej formie jest zawodnik, co w sportach zespołowych pomaga określić który zawodnik ma zagrać a który ma być rezerwowym.

Jako ostatni przykład podam badanie zmian klimatu. Częste zmiany minimum/maksimum temperatury na danym terenie powinny być sygnałem ostrzegawczym dla badaczy że dochodzi do lokalnego/globalnego ocieplenia/ochłodzenia o którym się dowiedzą, zanim te zmiany będą odczuwalne „na własnej skórze”.

2. Accumulate

Akumulacja jest operacją jeszcze bardziej przydatną ze względu na to, że można przekazać jaki operator binarny ma być aplikowany do elementów.

Pierwszym praktycznym zastosowaniem `std::accumulate` jest podsumowanie sprzedaży - obliczamy sumę sprzedaży z różnych transakcji w sklepie dzięki czemu można z niewielkim nakładem kodu wyliczyć dochody w danym okresie czasowym.

Drugim możliwym sposobem użycia tej metody jest algorytm konkatencji stringów, gdzie operatorem też jest plus tylko zaaplikowany do `std::string`.

Może to się przydać kiedy chcemy stworzyć jakiś szablon który ma pewną część stałą i kilka miejsc do manualnego wypełnienia. Byłoby to przydatne np. do automatycznego wysyłania spersonalizowanych maili, zwłaszcza tych reklamowych.

Trzecim zastosowaniem jest kryptografia – można stworzyć szyfrowanie, w którym użyjemy operatora XOR wraz z jakimś kluczem.

3. Merge

Scalanie 2 posortowanych struktur jest bardzo często spotykaną operacją.

W dodatku można ją połączyć z akumulacją aby otrzymać scalanie dowolnej ilości struktur. Najpopularniejszym zastosowaniem takiego wielokrotnego scalania są wszelkiego rodzaju rankingi, np. ranking międzyklasowego konkursu z matematyki. Ochotnicy z różnych klas biorą udział w konkursie, a potem ich prace są sprawdzane przez 3 różnych nauczycieli którzy na koniec łączą swoje wyniki w jedną listę aby określić kto wygrał konkurs.

Drugim zastosowaniem jest proces wyborów do sejmu – najpierw określa się kolejność kandydatów wewnątrz tego samego komitetu oraz ilość mandatów, które on zdobył, a później na podstawie tego można scalić wyniki każdego komitetu w 1 listę kandydatów, którzy zostali posłami.

Ostatnim zastosowaniem są internetowe repozytoria – każdy pull request posiada uporządkowaną listę plików, które się zmieniły, a następnie próbuje je połączyć z dotychczasowymi plikami repozytorium. Fakt, że dodawanie nowych zmian jest pewną odmianą scalania 2 list jest szczególnie widoczny gdy dodajemy nowe pliki w pull requestcie.

4. Sort

O zastosowaniach sortowania można by napisać osobną pracę dyplomową, ja przedstawię tylko trzy najczęściej spotykane.

Pierwszym z nich jest sortowanie produktów według danego kryterium (najczęściej cena) w internetowym sklepie. Jest to jedno z najpopularniejszych zastosowań w całej tej pracy, z którym na pewno każdy się spotkał.

Drugą sytuacją, w której sortowanie jest obecne i ułatwia życie jest lista kontaktów na telefonie. Ciężko jest wyobrazić sobie sytuację, w której mamy kilkadziesiąt albo i więcej kontaktów i są one w losowej kolejności, byłoby to bardzo niepraktyczne.

Ostatnim przykładem możliwego zastosowania sortowania jest kolejność plików w folderze. Tutaj, podobnie jak z kontaktami telefonicznymi, dobrze jest mieć pliki posortowane, np. alfabetycznie.

5. Transform

Transformacja danych wejściowych może być przydatna, kiedy chcemy stworzyć zmodyfikowaną kopię oryginału, np. poprzez konwersję do małych liter.

To ma zastosowanie przy wyszukiwaniu ilości wystąpień podstawa w słowie kiedy rozmiar czcionki nie ma znaczenia.

Drugie zastosowanie to suma dwóch wektorów, gdzie dodajemy do siebie kolejne elementy obu wektorów, co jest przydatne przy przetwarzaniu obrazów.

Ostatnim przykładem jest zmiana formatu dat. Otrzymujemy listę dat w formacie DD-MM-YY i chcemy przekształcić każdą z nich do DD.MM.YYYY

6. Unique

Może usuwać nie tylko sąsiadujące duplikaty (czyli wartości równe według operatora==) ale również sąsiadujące wartości dla których niestandardowy predykat zwraca true.

Przedstawię zastosowania przy założeniu, że dane są posortowane.

Pierwszym przykładem jest oczywiście usuwanie duplikatów, które w niektórych kontekstach i tak byłyby nadmiarowe i nie wносиły dodatkowych informacji.

Drugi przykład to zliczanie unikalnych elementów. Jeśli użyjemy `std::unique` jako drugi argument `std::distance`, gdzie pierwszym jest iterator na początek struktury, to dostaniemy ilość unikalnych elementów.

Ostatnim zastosowaniem może być usunięcie nadmiarowych logów wyprodukowanych przez program. Może się okazać, że program generuje kilka logów ostrzegających o jakimś błędzie i za pomocą unique i niestandardowego predykata będziemy w stanie przefiltrować te logi tak, aby każdy błąd miał 1 log.

7. N-th element

Pierwsze zastosowanie to szybkie znalezienie mediany, szybsze niż gdybyśmy użyli sortowania. Mediana jest często bardziej miarodajną oceną danego zbioru niż średnia, np. mediana zarobków lepiej odwzorowuje zarobki „przeciętnego” obywatela niż średnia zarobków, która jest zawyżona przez niewielką grupę bardzo bogatych osób.

Drugie zastosowanie to wybór n najlepszych elementów kiedy ich kolejność nie ma znaczenia. Przydatne gdy chcemy szybko dowiedzieć się, kto się dostał na jakiś kierunek studiów z dużą ilością ochotników.

Ostatnim przykładem jest wyliczenie centyla, co również jest przydatne w statystyce.

8. Regex

Walidacja danych w arkuszu internetowym może korzystać z regexa, np. email musi zawierać @.

Wydobywanie danych z ciągu – przydatne w aplikacjach do analizy tekstowej, np. możemy chcieć znaleźć wszystkie wystąpienia które są numerem telefonicznym.

Ostatni przykład to zamiana formatowania – np. wyszukujemy w tekście wszystkie wystąpienia daty w formacie DD-MM-YY i zamieniamy je na DD.MM.YY

9. Remove Erase if

Można użyć go przede wszystkim do filtrowania danych na podstawie jakiegoś warunku, do usunięcia z bazy danych wszystkich elementów o jakiejś wartości danego parametru, np. produktów które są na wyczerpaniu w przypadku sklepów internetowych.

Drugie zastosowanie to usunięcie kont wszystkich użytkowników, którzy są nieaktywni od jakiegoś czasu. W dzisiejszych czasach rzadziej spotykane, zwłaszcza w przypadku wielkich firm, ale i tak może to być przydatna opcja dla mniejszych graczy na rynku.

Ostatnie zastosowanie to automatyczne usuwanie komentarzy/postów które predykat uzna za łamiące zasady tego medium społecznościowego.

10. Generate

Pierwsze możliwe zastosowanie to generator losowych haseł, które byłyby silnymi hasłami do pierwszego użycia na danej stronie internetowej.

Drugie zastosowanie to generowanie sekwencji matematycznych, np. ciągu liczb pierwszych.

Ostatnie zastosowanie to tworzenie unikalnych identyfikatorów, które z jakiegoś powodu nie mogą być kolejnymi liczbami naturalnymi. Wtedy generator otrzymuje za zadanie wygenerować n liczb naturalnych które dotychczas nie zostały użyte (w określeniu czego pomaga mu stan wewnętrzny).

8. Wnioski końcowe

Jak było widać na wykresach własne implementacje były co najwyżej równie szybkie co wersje STL/Boost. Stąd pierwszy wniosek – nie warto „tworzyć koła na nowo”, jeśli jakiś algorytm istnieje w bibliotece STL lub Boost to lepiej go wykorzystać – oszczędzimy czas na stworzenie własnej implementacji, będziemy mieli co najmniej równie dobry czas wykonania (albo lepszy), będzie on bardziej elastyczny oraz zmniejszy ilość kodu.

Jeśli chodzi o wybór między STL a Boost, tutaj z reguły są one identyczne, jednak są sytuacje gdy jedna z nich jest lepsza. Wersję Boosta należy preferować w 2 głównych sytuacjach – po pierwsze gdy nie ma równoważnych metod w STL (np. algorytmy grafowe) a po drugie gdy wersja Boost nie wywołuje tej STL-owej. Wtedy to oznacza, że wersja Boost jest lepsza, gdyż została ona zaimplementowana później, kiedy znano wady wersji STL (jak w przypadku Regexa). STL z kolei jest lepszy w 2 sytuacjach – kiedy chcemy wykonać algorytm tylko na części struktury oraz jeśli nie chcemy/możemy pobrać biblioteki Boost z identyczną implementacją, co zwłaszcza w niedużych projektach może się nie opłacać.

Tabelka określająca kiedy warto korzystać z danej wersji algorytmu:

STL	Boost	Własna implementacja
Wykonujemy operacje na części struktury	STL nie ma równoważnej metody	Tylko jeśli STL/Boost nie ma odpowiednika
Potrzebujemy oszczędzić pamięć nie instalując Boosta	STL ma podobną, ale gorszą implementację	

Mateusz Mazur